Jawaharlal Nehru Technological University Hyderabad
University College of Engineering, Science & Technology

Real-Time Research Project Report

On

ResumeIQ: Semantic AI Resume Screening and Candidate Ranking System

Submitted By

A. SATHVIK - 23011M2110
SHIVA SATVIK JAKKAM - 23011M2111
CH. GANESH - 23011M2207

Department of Computer Science and Engineering
JNTUH University College of Engineering, Science & Technology
Kukatpally, Hyderabad - 500 085

Academic Year 2025-2026

DECLARATION BY THE CANDIDATES

We hereby declare that the project report entitled ResumeIQ: Semantic AI Resume Screening and Candidate Ranking System is prepared as a record of project work carried out by us. The system, documentation, testing notes, and explanation are based on our implementation of a local semantic AI resume screening application using Flask, SQLite, NLP preprocessing, resume parsing, sentence embeddings, ranking logic, and recruiter analytics. The material presented in this report is written for academic submission and demonstration purposes.

Date: _____

Signature: _____

CERTIFICATE BY THE SUPERVISOR

This is to certify that the project report entitled ResumeIQ: Semantic AI Resume Screening and Candidate Ranking System is submitted by A. Sathvik, Shiva Satvik Jakkam, and Ch. Ganesh in partial fulfillment of academic project requirements. The work describes a practical recruitment assistance system with local semantic AI matching, candidate ranking, and analytics for recruiter decision support.

Date: _____

Signature: _____

CERTIFICATE BY THE HEAD OF THE DEPARTMENT

This is to certify that the project work entitled ResumeIQ: Semantic AI Resume Screening and Candidate Ranking System has been prepared under the Department of Computer Science and Engineering. The report explains the problem, system design, implementation, testing, results, and future scope of the project.

Date: _____

Signature: _____

ABSTRACT

In the current recruitment environment, organizations receive a large number of resumes for each job opening, making manual screening slow, repetitive, and vulnerable to inconsistency. ResumeIQ addresses this problem by providing a lightweight semantic AI recruiter assistance system that uploads resumes, extracts resume text, preprocesses content, identifies skills, compares each resume with a job requirement, ranks candidates, and produces recruiter-focused analytics from the uploaded resume set.

The application uses Flask for the backend, SQLite for storage, pdfplumber and python-docx for resume parsing, NLTK for text preprocessing, Sentence Transformers with the all-MiniLM-L6-v2 model for semantic embeddings, and cosine similarity for meaning-level comparison. Unlike keyword-only screening, the system can recognize conceptual similarity between expressions such as backend API development and building REST APIs with FastAPI. The ranking mechanism combines semantic relevance, skill overlap, experience relevance, and project relevance into a clear final match percentage.

ResumeIQ also includes a modern recruiter dashboard built with Flask templates, TailwindCSS, and Chart.js. The dashboard displays match distribution, semantic relevance distribution, top detected skills, domain distribution, shortlisted versus rejected counts, skill gaps, resume completeness, and an AI insight panel. Recruiters can create jobs, expand job details, upload multiple resumes, filter candidates, shortlist or reject candidates, and download a PDF report of shortlisted candidates. The system is designed to run locally, remain modular, and be easy to explain during academic evaluation.

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, faculty members, and the Department of Computer Science and Engineering for their guidance and support throughout the development of ResumeIQ. Their direction helped us refine the idea from a simple resume ranking tool into a more complete recruiter assistance system with parsing, semantic matching, skill analytics, and an explainable dashboard.

We also acknowledge the open-source communities behind Flask, SQLite, NLTK, Sentence Transformers, pdfplumber, python-docx, TailwindCSS, Chart.js, NumPy, pandas, scikit-learn, and ReportLab. These technologies made it possible to build a practical and locally executable system with modern NLP and semantic matching capabilities.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1: INTRODUCTION

1.1 Problem Statement

ResumeIQ is a recruiter-oriented semantic AI system that screens resumes against a job requirement. It is intentionally lightweight, locally executable, and modular so that the project can be understood during academic evaluation. The system parses resumes, cleans text, extracts skills, generates semantic embeddings, calculates similarity, ranks candidates, and visualizes recruiter analytics.

Recruiters often receive many resumes for a single opening, and manual screening becomes slow when every candidate must be compared against the same requirement. The difficulty becomes more serious when resumes use different layouts and different vocabulary for similar work. A keyword-only approach can miss a good candidate when the same concept is written in different words.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

- Manual screening consumes recruiter time.
- Keyword-only systems may miss semantic similarity.
- Analytics are needed to understand the applicant pool.

1.2 Motivation

ResumeIQ is a recruiter-oriented semantic AI system that screens resumes against a job requirement. It is intentionally lightweight, locally executable, and modular so that the project can be understood during academic evaluation. The system parses resumes, cleans text, extracts skills, generates semantic embeddings, calculates similarity, ranks candidates, and visualizes recruiter analytics.

The motivation for ResumeIQ is to demonstrate a practical AI-assisted recruitment tool that can run on a local machine. The system uses semantic embeddings to capture the meaning of resume text instead of depending only on exact matching. This makes the project stronger than a basic resume parser while still remaining easy to install and explain.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

- Reduce repetitive resume review.
- Show local semantic AI in an academic project.
- Provide recruiter-friendly metrics instead of raw technical scores.

1.3 Objectives

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

The main objective is to upload resumes, parse content, preprocess text, extract skills, compare resumes with job requirements, rank candidates, shortlist suitable applicants, and generate analytics. The project also aims to keep the architecture simple, readable, and suitable for viva explanation.

A secondary objective is to support presentation demonstrations using 100 generated visual resumes across different fields. These resumes help show that relevant candidates rank higher than unrelated profiles.

- Create recruiter login and dashboard.
- Support PDF and DOCX upload.
- Generate semantic and skill-based scores.
- Display charts and insights from uploaded resumes.

1.4 Scope and Boundaries

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

The project scope is recruiter-side screening only. There is no candidate portal, interview scheduling, email automation, or enterprise applicant tracking integration. This keeps the project focused on semantic matching, ranking, and analytics.

The system is intended as a lightweight academic prototype that demonstrates a complete screening workflow rather than a commercial HR platform.

- In scope: recruiter dashboard, uploads, parsing, ranking, analytics.
- Out of scope: candidate accounts, payment systems, cloud deployment, email campaigns.

CHAPTER 2: REQUIREMENT ANALYSIS

2.1 Functional Requirements

ResumeIQ is a recruiter-oriented semantic AI system that screens resumes against a job requirement. It is intentionally lightweight, locally executable, and modular so that the project can be understood during academic evaluation. The system parses resumes, cleans text, extracts skills, generates semantic embeddings, calculates similarity, ranks candidates, and visualizes recruiter analytics.

Functional requirements define what the application must do. ResumeIQ must allow a recruiter to register, log in, create jobs, upload resumes, view rankings, filter candidates, shortlist or reject candidates, and export shortlisted candidates as a PDF report.

The system must also parse resume content, store extracted text, detect skills, compute scores, and generate analytics from the selected job only.

- Registration and login.
- Job creation, expansion, and deletion.
- Multiple resume upload.
- Ranking, filtering, shortlisting, rejection, and report export.

2.2 Non-Functional Requirements

Non-functional requirements describe system qualities. ResumeIQ should be lightweight, responsive, readable, modular, and locally executable. The user interface should be clean enough for repeated recruiter use and the code should be understandable for academic review.

The application should use recruiter-friendly percentages and clear labels. It should avoid unnecessary architecture complexity and should work on CPU without GPU-specific assumptions.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

- Local execution.
- Readable modules.
- Responsive dashboard.
- Clear metrics and charts.
- CPU-friendly semantic model.

2.3 Software Requirements

ResumeIQ uses Python and Flask for the backend, SQLite for database storage, pdfplumber and python-docx for parsing, NLTK for preprocessing, Sentence Transformers for semantic embeddings, and Chart.js for analytics visualization. TailwindCSS provides a modern dashboard style.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

The chosen libraries are open-source and suitable for a student project because they avoid heavy infrastructure while still providing real NLP and semantic matching capability.

2.4 Hardware Requirements

The system can run on a normal laptop with a dual-core processor, 8 GB RAM, and enough disk space for the virtual environment, model cache, database, uploaded resumes, and generated PDFs. A browser is required for using the dashboard.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

The first setup requires internet access for package and model download. After the model is cached, the backend matching process can operate locally.

2.5 Feasibility Study

Technical feasibility is strong because the application is built on mature Python libraries. Operational feasibility is also strong because recruiters interact through a simple dashboard. Economic feasibility is strong because all core technologies are free and open-source.

The main risk is text extraction quality for highly graphical resumes. This is acceptable for the project scope because the included visual resumes are text-based PDFs that can be parsed correctly.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

- Technical feasibility: high.
- Operational feasibility: high.
- Economic feasibility: high.
- Risk: image-only resumes require OCR in future.

CHAPTER 3: SYSTEM DESIGN

3.1 High Level Architecture

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

The architecture follows a simple Flask monolith pattern. Browser requests reach Flask routes in app.py. Parsing and preprocessing are delegated to utility modules. Semantic matching and analytics are delegated to AI modules. SQLite stores persistent data.

This design is easy to trace from a user action to a database change, which is valuable during project explanation.

- Browser -> Flask routes -> Utility modules -> AI modules -> SQLite -> Dashboard.

Fig. 3.1 High Level System Architecture
Recruiter Browser -> Flask Routes -> Utility Modules -> AI Modules -> SQLite -> Analytics Dashboard

Fig. 3.1: High level system architecture.

3.2 Recruiter Workflow

The recruiter workflow begins with authentication. After login, the recruiter creates a job requirement, uploads resumes, waits for analysis, reviews ranked candidates, checks analytics, and marks candidates as shortlisted or rejected.

The workflow is intentionally linear and simple. A recruiter can demonstrate the entire system within a few minutes by uploading one of the prepared presentation cases.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

- Login.
- Create job.
- Upload resumes.
- Analyze.
- Rank.
- Shortlist/reject.
- Download report.

3.3 Data Flow Design

Resume files enter the system through the upload form. Each file is validated, saved locally, parsed into text, cleaned, analyzed, and stored. The candidate table then reads stored scores and sorts by final match percentage.

The data flow separates raw resume text from cleaned text and score data. This makes the system easier to debug because the recruiter can trace whether a result came from parsing, skill extraction, semantic scoring, or ranking.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

3.4 Use Case Design

The primary actor is the recruiter/admin. The recruiter manages all actions: create jobs, upload resumes, review analytics, shortlist, reject, and export reports. The system actor performs automated parsing, preprocessing, skill extraction, semantic matching, ranking, and aggregation.

There is no candidate actor. This keeps the project aligned with the requirement that only recruiter/admin exists.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

- Actor: Recruiter/Admin.
- System responsibility: automated screening and analytics.
- No candidate portal.

3.5 Database Design

The database contains users, jobs, resumes, and candidates. The users table stores login records. The jobs table stores job descriptions and required skills. The resumes table stores raw text, cleaned text, contact details, skills, and sections. The candidates table stores match scores and recruiter decisions.

SQLite is selected because it is simple, local, and easy to explain. The project does not require a separate database server.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

Table	Purpose
users	Recruiter login records
jobs	Job descriptions and required skills
resumes	Uploaded resume text and extracted details
candidates	Scores, skill gaps, and status decisions

3.6 Security Design

Security is kept simple and appropriate for a local academic prototype. Passwords are hashed using Werkzeug helpers. Routes that require recruiter access are protected by a login_required decorator. Job and candidate actions verify ownership using the current session user.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

The system avoids exposing candidate actions to unauthenticated users. It also uses secure_filename before storing uploaded files.

- Password hashing.
- Session-based login.
- Route protection.
- Ownership checks.
- Secure filenames.

CHAPTER 4: SEMANTIC AI AND NLP METHODOLOGY

4.1 Resume Parsing

PDF resumes are parsed using pdfplumber. DOCX resumes are parsed using python-docx by reading paragraphs and table cells. Parsing converts documents into raw text so the rest of the system can operate on a common representation.

The parser extracts email and phone using regular expressions. Candidate names are inferred from early resume lines and cleaned when visual section labels appear near the name.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

4.2 NLP Preprocessing

NLP preprocessing lowercases text, removes punctuation, removes stopwords, tokenizes words, and lemmatizes tokens when resources are available. The cleaned output is more consistent for downstream matching.

Fallback stopwords and regex tokenization are included so that the application can continue working when NLTK resources are missing in offline mode.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

- Lowercase conversion.
- Whitespace normalization.
- Punctuation removal.
- Stopword removal.
- Tokenization.
- Lemmatization.

4.3 Skill Extraction

Skill extraction is dictionary-based and transparent. It includes categories for programming, frameworks, databases, cloud, tools, AI/data, design, marketing, business, HR/operations, finance/sales, support/QA, and infrastructure.

Regex matching is used so that skills are detected as meaningful terms rather than arbitrary substrings. Alias normalization improves consistency for common technology variations such as React Js and Node Js.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

- Keyword dictionaries.
- Regex boundary checks.
- Skill aliases.
- Overlap skills.
- Missing skills.

4.4 Semantic Embeddings

Sentence Transformers converts resume text and job descriptions into dense vectors. ResumeIQ uses all-MiniLM-L6-v2 because it is lightweight and suitable for CPU execution. The embedding captures semantic meaning beyond exact keywords.

The model is loaded lazily so the Flask app can start before the first semantic analysis request. Local cache is tried first, which helps the system run consistently once the model has been downloaded.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

4.5 Cosine Similarity

Cosine similarity compares two embedding vectors. Higher similarity means the resume and job description are closer in meaning. ResumeIQ uses this for overall resume relevance, experience relevance, and project relevance.

The raw similarity value is converted into percentages in the dashboard so recruiters do not need to interpret decimal vector scores.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

4.6 Ranking Formula

The ranking formula combines semantic similarity, skill match, experience relevance, and project relevance. Semantic similarity contributes 40 percent, skill match contributes 35 percent, experience relevance contributes 15 percent, and project relevance contributes 10 percent.

This formula is explainable and adjustable. It gives importance to both meaning-level fit and explicit required skills.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

- Semantic: 40%.
- Skill match: 35%.
- Experience: 15%.
- Projects: 10%.

Signal	Weight
Semantic similarity	40%
Skill match	35%
Experience relevance	15%
Project relevance	10%

4.7 Resume Completeness Score

Resume completeness is calculated using rule-based signals: sufficient text length, number of detected skills, presence of experience, projects, education, certifications, email, and phone number.

The completeness score is not the same as job fit. It indicates resume quality and structure. A resume can be complete but not relevant, or relevant but poorly structured.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

CHAPTER 5: IMPLEMENTATION

5.1 Flask Routes

The main Flask routes handle home redirection, registration, login, logout, dashboard rendering, job creation, job deletion, resume upload, status updates, and shortlisted candidate PDF download.

Keeping these routes in app.py makes the academic flow easy to follow. Each route calls focused helper functions for parsing, scoring, and data access.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

Route	Purpose
/dashboard	Main recruiter dashboard
/jobs	Create job
/upload/	Upload and analyze resumes
/candidate//status	Shortlist or reject
/jobs//selected-report.pdf	Download shortlist report

5.2 AI Modules

The ai folder contains embeddings.py, matcher.py, ranking.py, and analytics.py. Embeddings loads the Sentence Transformer model. Matcher calculates cosine similarity. Ranking applies the weighted formula. Analytics prepares metric cards, chart data, and recruiter insights.

This separation prevents the Flask routes from becoming too complex and keeps AI logic isolated for testing.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

5.3 Utility Modules

The utils folder contains parser.py, preprocess.py, skills.py, scoring.py, and database.py. These modules handle document parsing, NLTK preprocessing, skill extraction, rule-based resume strength, and SQLite access.

The utility modules use readable functions rather than hidden framework behavior, which supports the project goal of being beginner-friendly and viva-explainable.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

5.4 Dashboard Implementation

The dashboard uses Flask templates and TailwindCSS. The top area contains job creation, the scrollable job list with details expanders, and resume upload. The analytics area contains metric cards and charts. The candidate table spans the full width for readability.

Chart.js renders match distribution, semantic distribution, top skills, domain distribution, and status distribution. The JavaScript is minimal and receives chart data from Flask as JSON.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

5.5 Shortlisted PDF Export

The shortlisted report route queries candidates marked as Shortlisted for the selected job and uses ReportLab to produce a PDF. The PDF contains rank, name, match percentage, semantic percentage, skill match percentage, and email.

This feature supports the recruiter workflow by allowing the final selected list to be shared or stored separately from the dashboard.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

5.6 Generated Visual Resumes

The project includes 100 generated visual resumes in PDF format. They cover backend, frontend, full stack, data science, machine learning, NLP, DevOps, cloud, cybersecurity, database, mobile, design, marketing, HR, business, project, product, finance, sales, support, content, QA, and operations roles.

These resumes make it possible to test analytics and demonstrate ranking behavior without requiring private real candidate resumes.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

CHAPTER 6: ANALYTICS AND METRICS

6.1 Recruiter Metrics

The analytics dashboard includes total resumes, average match, average semantic score, average skill match, shortlisted count, top skill, most missing skill, average completeness, top candidate, experience relevance, and project relevance.

All metrics are calculated from database records for the selected job. This keeps the dashboard honest and avoids fake or random analytics.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

6.2 Chart Design

Chart.js visualizes the applicant pool. Match distribution and semantic distribution use score ranges from 0-20 percent through 80-100 percent. Top detected skills use a horizontal bar chart with frequency counts. Domain and status charts use doughnut charts.

The charts translate technical analysis into recruiter-readable patterns. A recruiter can quickly see whether most candidates are weak, moderate, or strong for the job.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

6.3 AI Insight Panel

The AI insight panel converts aggregate metrics into short recruiter observations. Example insights include the dominant applicant domain, most missing skill, percentage above 70 percent match, and whether average semantic relevance is high, moderate, or low.

The insights are dynamically generated from uploaded resume data for the selected job, not from fixed placeholder text.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

CHAPTER 7: TESTING

7.1 Unit Testing

Unit tests verify parser behavior, preprocessing, skill extraction, scoring, analytics, routes, and PDF report generation. These tests make sure the important modules work independently.

Skill alias tests are especially important because recruiter input may use React Js, Next Js, Express Js, or Node Js while resumes use different spelling.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

7.2 Integration Testing

Integration tests use Flask test clients to register a recruiter, create jobs, upload resumes, and verify dashboard behavior. This confirms that modules work together through real routes.

The app also includes tests for PDF report download so the shortlisted candidate export is covered.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

7.3 Full Pipeline Testing

The full pipeline check uploads all 100 generated visual resumes into a temporary database, creates candidate records, calculates scores, generates charts, and checks summary values.

The observed full pipeline confirms 100 uploaded resumes and 100 created candidates for the backend demo job. This proves the generated resumes are compatible with the parser and matching pipeline.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

7.4 Offline Testing

Offline testing uses cached model resources and environment variables to ensure the backend can process resumes without network access after first setup. The project also includes a comparison script that verifies offline and normal cached summaries match.

This is useful for live presentations where internet access may be unstable.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

CHAPTER 8: USER GUIDE

8.1 Setup Summary

The user creates a Python virtual environment, installs requirements, runs python app.py, and opens http://127.0.0.1:5001 in a browser. The README contains detailed macOS, Linux, and Windows instructions.

The first setup requires package and model download. After that, local cached execution can be used for analysis and tests.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

8.2 Recruiter Workflow

The recruiter registers, logs in, creates a job requirement, uploads resumes, checks the ranked candidate table, reviews analytics, shortlists or rejects candidates, and downloads the selected candidate report.

The job list includes expandable details so the recruiter can verify job descriptions and required skills even when many jobs are stored.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

8.3 Presentation Workflow

For presentation, the recommended cases are Backend API Hiring, Data Science Analytics Hiring, and DevOps Cloud Hiring. These cases show clear domain ranking and useful analytics.

The presenter should create a fresh job for each case and upload only the listed six PDFs from PRESENTATION_INPUTS.md.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

CHAPTER 9: RESULTS AND DISCUSSION

9.1 Backend API Result

In the backend API case, Aarav Sharma ranks first because the resume overlaps strongly with Python, Flask, FastAPI, PostgreSQL, SQLite, and Git. This demonstrates correct ranking for a technical backend role.

Average scores are lower than the top score because unrelated resumes are intentionally included in the upload set. This helps the analytics show a realistic quality distribution.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

9.2 Data Science Result

In the data science analytics case, data analyst, data scientist, machine learning, and NLP resumes rank higher than unrelated resumes. This demonstrates semantic matching across related technical domains.

The expected top candidate is Sia Bakshi, with Rudra Talwar also ranking high. The top skill is often Python because it appears across data-oriented resumes.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

9.3 DevOps Cloud Result

In the DevOps cloud case, Sneha Sarin and Kartik Patil rank closely. This is a useful result because both cloud and DevOps profiles are genuinely relevant to AWS, Docker, Kubernetes, Linux, Terraform, and CI/CD requirements.

The ranking shows that ResumeIQ can handle related but non-identical domains and still put the right candidates near the top.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

9.4 Score Interpretation

The score should be treated as decision support, not as an automatic hiring decision. A high score means the candidate is likely relevant, but the recruiter should still review actual experience, projects, and resume details.

Small score changes are normal if the job description changes. The important demonstration point is that relevant resumes rank above unrelated resumes.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

CHAPTER 10: LIMITATIONS AND FUTURE SCOPE

10.1 Limitations

ResumeIQ depends on text extraction quality. If a resume is scanned as an image, pdfplumber may not extract text without OCR. Section extraction is regex-based and can miss unusual headings. Skill dictionaries must be expanded over time for new technologies.

The project also does not include candidate accounts, interview workflows, email automation, or enterprise ATS integrations.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

10.2 Future Scope

Future enhancements can include OCR, richer skill dictionaries, recruiter notes, CSV export, resume preview, interview pipeline tracking, role templates, and local asset bundling for fully offline UI rendering.

Analytics can also be extended with historical hiring trends, job-to-job comparisons, and saved benchmark reports.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

10.3 Conclusion

ResumeIQ successfully demonstrates a local semantic AI resume screening and candidate ranking system. It integrates parsing, preprocessing, skill extraction, semantic embeddings, cosine similarity, weighted ranking, and recruiter analytics into one practical Flask application.

The project is useful for academic demonstration because the architecture is simple, the scoring is explainable, and the outputs are visible through a recruiter-focused dashboard.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

APPENDIX A: INSTALLATION SUMMARY

A.1 macOS Setup

On macOS, install Homebrew, install Python, create a virtual environment, install requirements, and run python app.py. The dashboard opens at 127.0.0.1:5001.

The README gives exact command sequences for Homebrew, Python, pip, virtual environment activation, dependency installation, and app startup.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

A.2 Linux Setup

On Ubuntu or Debian, install python3, python3-pip, and python3-venv. Then create a virtual environment, install requirements, and run python app.py.

Other Linux distributions can use their own package managers as long as Python, pip, and venv are available.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

A.3 Windows Setup

On Windows, install Python from python.org and enable Add Python to PATH. Then use PowerShell to create and activate a virtual environment, install requirements, and run the app.

If PowerShell blocks script execution, set the current user execution policy to RemoteSigned and activate the environment again.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

APPENDIX B: DATA DICTIONARY

B.1 Users and Jobs Tables

The users table stores recruiter login details, while the jobs table stores job requirements. Each job belongs to one recruiter and contains a title, description, required skills, and timestamp.

This design allows one recruiter account to maintain multiple job requirements in the same local database.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

B.2 Resumes Table

The resumes table stores the uploaded filename, stored path, raw text, cleaned text, extracted email, phone, skills, and serialized sections. Keeping both raw and cleaned text helps debugging and explanation.

The sections field stores extracted experience, projects, education, and certifications in JSON form.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

B.3 Candidates Table

The candidates table stores all score components and recruiter decisions. It includes semantic score, skill score, experience score, project score, resume strength, final score, overlap skills, missing skills, and status.

This table powers the ranked candidate display, status chart, shortlisted report, and many analytics cards.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

APPENDIX C: DEMO CASES

C.1 Backend API Hiring

Upload Aarav Sharma, Rohan Pillai, Nisha Varma, Arjun Raman, Kabir Chatterjee, and Priya Soman. Aarav Sharma should rank first with a score near 67.69 percent.

This case demonstrates that backend skills and API experience produce a stronger match than unrelated design or data science profiles.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

C.2 Data Science Analytics Hiring

Upload Sia Bakshi, Rudra Talwar, Kiara Bedi, Neil Gandhi, Ina Ojha, and Aditi Fernandes. Sia Bakshi should rank first and Rudra Talwar should rank high.

This case demonstrates data-oriented ranking using Python, SQL, pandas, machine learning, NLP, Tableau, and Power BI signals.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

C.3 DevOps Cloud Hiring

Upload Kartik Patil, Sneha Sarin, Ansh Sawant, Farah Saha, Samira Ghosh, and Suhani Rastogi. Sneha Sarin and Kartik Patil should rank close to the top.

This case demonstrates infrastructure ranking and domain distribution for cloud and DevOps profiles.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

APPENDIX D: VIVA EXPLANATION

D.1 Why Sentence Transformers?

Sentence Transformers provide dense semantic embeddings that represent the meaning of text. This helps ResumeIQ compare resumes and job descriptions even when wording differs.

The selected all-MiniLM-L6-v2 model is small, fast, and suitable for CPU execution, making it appropriate for a local academic project.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.

D.2 Why SQLite?

SQLite is simple, file-based, and does not require a separate database server. For a local academic project, this reduces setup complexity while still supporting structured tables and SQL queries.

The schema is also easy to explain because users, jobs, resumes, and candidates map directly to project concepts.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

D.3 Why Flask?

Flask provides simple routing, templates, request handling, sessions, and integration with Python modules. It allows ResumeIQ to be built as one understandable application rather than a complex multi-service system.

This makes the project easier to debug, demonstrate, and explain in a viva.

The application stores data locally using SQLite and uploaded files in the uploads folder. Semantic matching is performed in Python using Sentence Transformers and cosine similarity. This keeps the architecture understandable while still demonstrating modern NLP and semantic AI concepts.

APPENDIX E: REFERENCES

E.1 References

Flask documentation for routing and templates. SQLite documentation for local relational storage. NLTK documentation for preprocessing resources. Sentence Transformers documentation for all-MiniLM-L6-v2 embeddings. pdfplumber documentation for PDF extraction. python-docx documentation for DOCX extraction. Chart.js documentation for browser charts. TailwindCSS documentation for styling. ReportLab documentation for PDF creation.

The project also draws conceptual background from recruitment screening workflows, semantic similarity, cosine similarity, NLP preprocessing, and explainable ranking systems.

The key value of the project is that it does more than list resumes. It explains candidate quality through match percentage, semantic relevance, skill coverage, missing skills, experience relevance, project relevance, resume completeness, and status-based recruiter decisions. This turns resume screening from a manual reading task into an assisted decision workflow.